

虽然模板在工程中的应用范围很广，而且功能十分强大^①，但选手们却很少会在算法竞赛中亲自编写模板。那为什么还要介绍模板呢？主要是因为模板有助于读者更好地理解 STL。

5.2 STL 初步

STL 是指 C++ 的标准模板库（Standard Template Library）。它很好用，但也很复杂。本节将介绍 STL 中的一些常用算法和容器，在后面的章节中还会继续介绍本节没有涉及的其他内容。

5.2.1 排序与检索

例题 5-1 大理石在哪儿（Where is the Marble?, UVa 10474）

现有 N 个大理石，每个大理石上写了一个非负整数。首先把各数从小到大排序，然后回答 Q 个问题。每个问题问是否有一个大理石写着某个整数 x ，如果是，还要回答哪个大理石上写着 x 。排序后的大理石从左到右编号为 $1 \sim N$ 。（在样例中，为了节约篇幅，所有大理石上的数合并到一行，所有问题也合并到一行。）

样例输入：

```
4 1
2 3 5 1
5
5 2
1 3 3 3 1
2 3
```

样例输出：

```
CASE# 1:
5 found at 4
CASE# 2:
2 not found
3 found at 3
```

【分析】

题目意思已经清楚了：先排序，再查找。使用 `algorithm` 头文件中的 `sort` 和 `lower_bound` 很容易完成这两项操作，代码如下：

```
#include<cstdio>
#include<algorithm>
using namespace std;
```

^① 有兴趣的读者可以研究一下 C++ 的模板元编程（template metaprogramming）。在 boost 库中有很多模板元编程的优秀例子。

```

const int maxn = 10000;

int main() {
    int n, q, x, a[maxn], kase = 0;
    while(scanf("%d%d", &n, &q) == 2 && n) {
        printf("CASE# %d:\n", ++kase);
        for(int i = 0; i < n; i++) scanf("%d", &a[i]);
        sort(a, a+n); //排序
        while(q--) {
            scanf("%d", &x);
            int p = lower_bound(a, a+n, x) - a; //在已排序数组 a 中寻找 x
            if(a[p] == x) printf("%d found at %d\n", x, p+1);
            else printf("%d not found\n", x);
        }
    }
    return 0;
}

```

上面的代码比第4章中的排序代码简单很多，因为省略了一个 `compare` 函数——`sort` 使用数组元素默认的大小比较运算符进行排序，只有在需要按照特殊依据进行排序时才需要传入额外的比较函数。

另外，`sort` 可以对任意对象进行排序，不一定是内置类型。如果希望用 `sort` 排序，这个类型需要定义“小于”运算符，或者在排序时传入一个“小于”函数。排序对象可以存在于普通数组里，也可以存在于 `vector` 中（参见 5.2.2 节）。前者用 `sort(a, a+n)` 的方式调用，后者用 `sort(v.begin(), v.end())` 的方式调用。`lower_bound` 的作用是查找“大于或者等于 `x` 的第一个位置”。

为什么 `sort` 可以对任意对象进行排序呢？学习了前面内容，相信读者可以猜到，这是因为 `sort` 是一个模板函数。

提示 5-11：`algorithm` 头文件中的 `sort` 可以给任意对象排序，包括内置类型和自定义类型，前提是类型定义了“<”运算符。排序之后可以用 `lower_bound` 查找大于或等于 `x` 的第一个位置。待排序/查找的元素可以放在数组里，也可以放在 `vector` 里。

还有一个 `unique` 函数可以删除有序数组中的重复元素，后面的例题中将展示其用法。

5.2.2 不定长数组：vector

`vector` 就是一个不定长数组。不仅如此，它把一些常用操作“封装”在了 `vector` 类型内部。例如，若 `a` 是一个 `vector`，可以用 `a.size()` 读取它的大小，`a.resize()` 改变大小，`a.push_back()` 向尾部添加元素，`a.pop_back()` 删除最后一个元素。

`vector` 是一个模板类，所以需要 `vector<int> a` 或者 `vector<double> b` 这样的方式来声明一个 `vector`。`vector<int>` 是一个类似于 `int a[]` 的整数数组，而 `vector<string>` 就是一个类似

于 `string a[]` 的字符串数组。`vector` 看上去像是“一等公民”，因为它们可以直接赋值，还可以作为函数的参数或者返回值，而无须像传递数组那样另外用一个变量指定元素个数。

例题 5-2 木块问题 (The Blocks Problem, UVa 101)

从左到右有 n 个木块，编号为 $0 \sim n-1$ ，要求模拟以下 4 种操作（下面的 a 和 b 都是木块编号）。

- `move a onto b`: 把 a 和 b 上方的木块全部归位，然后把 a 摆在 b 上面。
- `move a over b`: 把 a 上方的木块全部归位，然后把 a 放在 b 所在木块堆的顶部。
- `pile a onto b`: 把 b 上方的木块全部归位，然后把 a 及上面的木块整体摆在 b 上面。
- `pile a over b`: 把 a 及上面的木块整体摆在 b 所在木块堆的顶部。

遇到 `quit` 时终止一组数据。 a 和 b 在同一堆的指令是非法指令，应当忽略。

样例输入：

1234

样例输出：

1234 -> 3087 -> 8352 -> 6174 -> 6174

【分析】

每个木块堆的高度不确定，所以用 `vector` 来保存很合适；而木块堆的个数不超过 n ，所以用一个数组来存就可以了。代码如下：

```
#include <cstdio>
#include <string>
#include <vector>
#include <iostream>
using namespace std;

const int maxn = 30;
int n;
vector<int> pile[maxn]; //每个 pile[i] 是一个 vector

//找木块 a 所在的 pile 和 height，以引用的形式返回调用者
void find_block(int a, int& p, int& h) {
    for(p = 0; p < n; p++)
        for(h = 0; h < pile[p].size(); h++)
            if(pile[p][h] == a) return;
}

//把第 p 堆高度为 h 的木块上方的所有木块移回原位
void clear_above(int p, int h) {
    for(int i = h+1; i < pile[p].size(); i++) {
        int b = pile[p][i];
```

```

    pile[b].push_back(b); //把木块b放回原位
}
pile[p].resize(h+1); //pile只应保留下标 0~h 的元素
}

//把第 p 堆高度为 h 及其上方的木块整体移动到 p2 堆的顶部
void pile_onto(int p, int h, int p2) {
    for(int i = h; i < pile[p].size(); i++)
        pile[p2].push_back(pile[p][i]);
    pile[p].resize(h);
}

void print() {
    for(int i = 0; i < n; i++) {
        printf("%d:", i);
        for(int j = 0; j < pile[i].size(); j++) printf(" %d", pile[i][j]);
        printf("\n");
    }
}

int main() {
    int a, b;
    cin >> n;
    string s1, s2;
    for(int i = 0; i < n; i++) pile[i].push_back(i);
    while(cin >> s1 >> a >> s2 >> b) {
        int pa, pb, ha, hb;
        find_block(a, pa, ha);
        find_block(b, pb, hb);
        if(pa == pb) continue; //非法指令
        if(s2 == "onto") clear_above(pb, hb);
        if(s1 == "move") clear_above(pa, ha);
        pile_onto(pa, ha, pb);
    }
    print();
    return 0;
}

```

数据结构的核心是 `vector<int> pile[maxn]`, 所有操作都是围绕它进行的。`vector` 就像一个二维数组, 只是第一维的大小是固定的 (不超过 `maxn`), 但第二维的大小不固定。上述代码还有一个值得学习的技巧: 输入一共有 4 种指令, 但如果完全独立地处理各指令, 代码就会变得冗长而且易错。更好的方法是提取出指令之间的共同点, 编写函数以减少重复代码。

提示 5-12: `vector` 头文件中的 `vector` 是一个不定长数组，可以用 `clear()` 清空，`resize()` 改变大小，用 `push_back()` 和 `pop_back()` 在尾部添加和删除元素，用 `empty()` 测试是否为空。`vector` 之间可以直接赋值或者作为函数的返回值，像是“一等公民”一样。

5.2.3 集合：set

集合与映射也是两个常用的容器。`set` 就是数学上的集合——每个元素最多只出现一次。和 `sort` 一样，自定义类型也可以构造 `set`，但同样必须定义“小于”运算符。

例题 5-3 安迪的第一个字典（Andy's First Dictionary, UVa 10815）

输入一个文本，找出所有不同的单词（连续的字母序列），按字典序从小到大输出。单词不区分大小写。

样例输入：

Adventures in Disneyland

Two blondes were going to Disneyland when they came to a fork in the road. The sign read: "Disneyland Left."

So they went home.

样例输出（为了节约篇幅只保留前 5 行）：

a
adventures
blondes
came
disneyland

【分析】

本题没有太多的技巧，只是为了展示 `set` 的用法：由于 `string` 已经定义了“小于”运算符，直接使用 `set` 保存单词集合即可。注意，输入时把所有非字母的字符变成空格，然后利用 `stringstream` 得到各个单词。

```
#include<iostream>
#include<string>
#include<set>
#include<sstream>
using namespace std;
```

```
set<string> dict; //string 集合
```

```
int main() {
    string s, buf;
    while(cin >> s) {
```

```

for(int i = 0; i < s.length(); i++)
    if(isalpha(s[i])) s[i] = tolower(s[i]); else s[i] = ' ';
stringstream ss(s);
while(ss >> buf) dict.insert(buf);
}
for(set<string>::iterator it = dict.begin(); it != dict.end(); ++it)
    cout << *it << "\n";
return 0;
}

```

上面的代码用到了 `set` 中元素已从小到大排好序这一性质，用一个 `for` 循环即可从小到大遍历所有元素。

代码里的 `set<string>::iterator` 是什么？`dict.begin()` 和 `dict.end()` 又是什么？`iterator` 的意思是迭代器，是 STL 中的重要概念，类似于指针。和“`vector` 类似于数组”一样，这里的“类似”指的是用法类似。还记得第 4 章中的那个 `sum` 函数吗？

```

int sum(int* begin, int* end) {
    int *p = begin;
    int ans = 0;
    for(int *p = begin; p != end; p++)
        ans += *p;
    return ans;
}

```

这个 `for` 循环是不是和上面的代码很像？实际上，上面参数中的 `begin` 和 `end` 就是仿照 STL 中的迭代器命名的。

5.2.4 映射：map

`map` 就是从键（key）到值（value）的映射。因为重载了 `[]` 运算符，`map` 像是数组的“高级版”。例如可以用一个 `map<string,int> month_name` 来表示“月份名字到月份编号”的映射，然后用 `month_name["July"] = 7` 这样的方式来赋值。

例题 5-4 反片语（Anagrams, UVa 156）

输入一些单词，找出所有满足如下条件的单词：该单词不能通过字母重排，得到输入文本中的另外一个单词。在判断是否满足条件时，字母不分大小写，但在输出时应保留输入中的大小写，按字典序进行排列（所有大写字母在所有小写字母的前面）。

样例输入：

```

ladder came tape soon leader acme RIDE lone Dreis peat
ScAlE orb eye Rides dealer NotE derail LaCeS drIed
noel dire Disk mace Rob dries
#

```

样例输出：

```
Disk
NotE
derail
drIed
eye
ladder
soon
```

【分析】

把每个单词“标准化”，即全部转化为小写字母后再进行排序，然后再放到 `map` 中进行统计。代码如下：

```
#include<iostream>
#include<string>
#include<cctype>
#include<vector>
#include<map>
#include<algorithm>
using namespace std;

map<string,int> cnt;
vector<string> words;

//将单词 s 进行“标准化”
string repr(const string& s) {
    string ans = s;
    for(int i = 0; i < ans.length(); i++)
        ans[i] = tolower(ans[i]);
    sort(ans.begin(), ans.end());
    return ans;
}

int main() {
    int n = 0;
    string s;
    while(cin >> s) {
        if(s[0] == '#') break;
        words.push_back(s);
        string r = repr(s);
        if(!cnt.count(r)) cnt[r] = 0;
        cnt[r]++;
    }
}
```

```

}
vector<string> ans;
for(int i = 0; i < words.size(); i++)
    if(cnt[repr(words[i])] == 1) ans.push_back(words[i]);
sort(ans.begin(), ans.end());
for(int i = 0; i < ans.size(); i++)
    cout << ans[i] << "\n";
return 0;
}

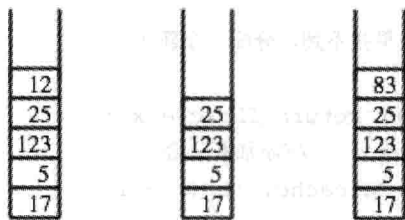
```

此例说明，如果没有良好的代码设计，是无法发挥 STL 的威力的。如果没有想到“标准化”这个思路，就很难用 map 简化代码。

提示 5-13: set 头文件中的 set 和 map 头文件中的 map 分别是集合与映射。二者都支持 insert、find、count 和 remove 操作，并且可以按照从小到大的顺序循环遍历其中的元素。map 还提供了“[]”运算符，使得 map 可以像数组一样使用。事实上，map 也称为“关联数组”。

5.2.5 栈、队列与优先队列

STL 还提供 3 种特殊的数据结构：栈、队列与优先队列。所谓栈，就是符合“后进先出”（Last In First Out, LIFO）规则的数据结构，有 PUSH 和 POP 两种操作，其中 PUSH 把元素压入“栈顶”，而 POP 从栈顶把元素“弹出”，如图 5-1 所示。



Original stack. After pop(). After push(83).

图 5-1 PUSH 和 POP 操作

讲一个有趣的笑话。如何判断一个人是不是程序员？答：问它 PUSH 的反义词是什么。回答 PULL 的是普通人，而回答 POP 的才是程序员^①。这个笑话间接地说明了“栈”这个数据结构在计算机中的重要性。

STL 的栈定义在头文件<stack>中，可以用“stack<int> s”方式声明一个栈。

提示 5-14: STL 的 stack 头文件提供了栈，用“stack<int> s”方式定义，用 push()和 pop()实现元素的入栈和出栈操作，top()取栈顶元素（但不删除）。

例题 5-5 集合栈计算机(The SetStack Computer, ACM/ICPC NWERC 2006, UVa12096)

有一个专门为了集合运算而设计的“集合栈”计算机。该机器有一个初始为空的栈，

^① 如果你想较真的话，这里有一个反例：经常使用 git 的程序员也有可能回答 pull。

并且支持以下操作。

- ❑ PUSH: 空集 “{}” 入栈。
 - ❑ DUP: 把当前栈顶元素复制一份后再入栈。
 - ❑ UNION: 出栈两个集合，然后把二者的并集入栈。
 - ❑ INTERSECT: 出栈两个集合，然后把二者的交集入栈。
 - ❑ ADD: 出栈两个集合，然后把先出栈的集合加入到后出栈的集合中，把结果入栈。
- 每次操作后，输出栈顶集合的大小（即元素个数）。例如，栈顶元素是 $A = \{\{\}, \{\{\}\}\}$ ，

下一个元素是 $B = \{\{\}, \{\{\{\}\}\}\}$ ，则：

- ❑ UNION 操作将得到 $\{\{\}, \{\{\}\}, \{\{\{\}\}\}\}$ ，输出 3。
- ❑ INTERSECT 操作将得到 $\{\{\}\}$ ，输出 1。
- ❑ ADD 操作将得到 $\{\{\}, \{\{\{\}\}\}, \{\{\}, \{\{\}\}\}\}$ ，输出 3。

输入不超过 2000 个操作，并且保证操作均能顺利进行（不需要对空栈执行出栈操作）。

【分析】

本题的集合并不是简单的整数集合或者字符串集合，而是集合的集合。为了方便起见，此处为每个不同的集合分配一个唯一的 ID，则每个集合都可以表示成所包含元素的 ID 集合，这样就可以用 STL 的 `set<int>` 来表示了，而整个栈则是一个 `stack<int>`。

```
typedef set<int> Set;
map<Set, int> IDcache; //把集合映射成 ID
vector<Set> Setcache; //根据 ID 取集合

//查找给定集合 x 的 ID。如果找不到，分配一个新 ID
int ID (Set x) {
    if (IDcache.count(x)) return IDcache[x];
    Setcache.push_back(x); //添加新集合
    return IDcache[x] = Setcache.size() - 1;
}
```

对任意集合 s （类型是上面定义的 `Set`），`IDcache[s]` 就是它的 ID，而 `Setcache[IDcache[s]]` 就是 s 本身。下面的 `ALL` 和 `INS` 是两个宏^①：

```
#define ALL(x) x.begin(), x.end()
#define INS(x) inserter(x, x.begin())
```

分别表示“所有的内容”以及“插入迭代器”，具体作用可以从代码中推断出来，有兴趣的读者可以查阅 STL 文档以了解更详细的信息。主程序如下，请读者注意 STL 内置的集合操作（如 `set_union` 和 `set_intersection`）。

```
stack<int> s; //题目中的栈
int n;
cin >> n;
```

^① 宏（macro）是一个很复杂的话题，这里读者暂时可以把带参数的宏理解为“类似于函数的东西”。

```

for(int i = 0; i < n; i++) {
    string op;
    cin >> op;
    if (op[0] == 'P') s.push(ID(Set()));
    else if (op[0] == 'D') s.push(s.top());
    else {
        Set x1 = Setcache[s.top()]; s.pop();
        Set x2 = Setcache[s.top()]; s.pop();
        Set x;
        if (op[0] == 'U') set_union (ALL(x1), ALL(x2), INS(x));
        if (op[0] == 'I') set_intersection (ALL(x1), ALL(x2), INS(x));
        if (op[0] == 'A') { x = x2; x.insert(ID(x1)); }
        s.push(ID(x));
    }
    cout << Setcache[s.top()].size() << endl;
}

```

本题极为重要，后面章节中有一些例题也使用了本题的解决方法，建议读者仔细体会。队列是符合“先进先出”（First In First Out, FIFO）原则的“公平队列”，无须过多介绍，如图 5-2 所示。

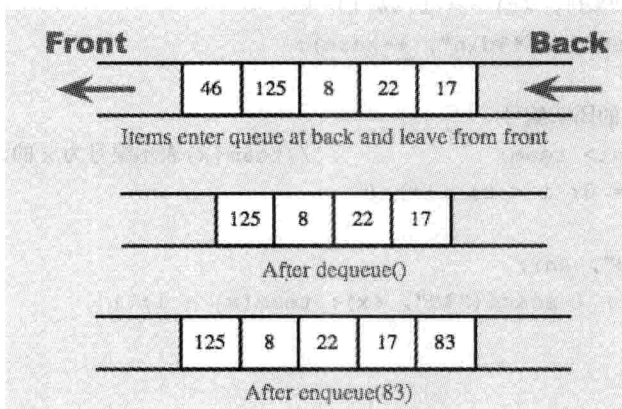


图 5-2 队列

STL 队列定义在头文件<queue>中，可以用“queue<int> s”方式声明一个队列。

提示 5-15: STL 的 queue 头文件提供了队列，用“queue<int> s”方式定义，用 push()和 pop()进行元素的入队和出队操作，front()取队首元素（但不删除）。

例题 5-6 团体队列（Team Queue, UVa 540）

有 t 个团队的人正在排一个长队。每次新来一个人时，如果他有队友在排队，那么这个新人会插队到最后一个队友的身后。如果没有任何一个队友排队，则他会排到长队的队尾。输入每个团队中所有队员的编号，要求支持如下 3 种指令（前两种指令可以穿插进行）。

- ENQUEUE x: 编号为 x 的人进入长队。
- DEQUEUE: 长队的队首出队。
- STOP: 停止模拟。

对于每个 DEQUEUE 指令, 输出出队的人的编号。

【分析】

本题有两个队列: 每个团队有一个队列, 而团队整体又形成一个队列。例如, 有 3 个团队 1, 2, 3, 队员集合分别为 {101, 102, 103, 104}、{201, 202} 和 {301, 302, 303}, 当前长队为 {301, 303, 103, 101, 102, 201}, 则 3 个团队的队列分别为 {103, 101, 102}、{201} 和 {301, 303}, 团队整体的队列为 {3, 1, 2}。代码如下:

```
#include<cstdio>
#include<queue>
#include<map>
using namespace std;

const int maxt = 1000 + 10;

int main() {
    int t, kase = 0;
    while(scanf("%d", &t) == 1 && t) {
        printf("Scenario #%d\n", ++kase);

        //记录所有人的团队编号
        map<int, int> team; //team[x]表示编号为 x 的人所在的团队编号
        for(int i = 0; i < t; i++) {
            int n, x;
            scanf("%d", &n);
            while(n--) { scanf("%d", &x); team[x] = i; }
        }

        //模拟
        queue<int> q, q2[maxt]; //q 是团队的队列, 而 q2[i] 是团队 i 成员的队列
        for(;;) {
            int x;
            char cmd[10];
            scanf("%s", cmd);
            if(cmd[0] == 'S') break;
            else if(cmd[0] == 'D') {
                int t = q.front();
                printf("%d\n", q2[t].front()); q2[t].pop();
                if(q2[t].empty()) q.pop(); //团体 t 全体出队列
            }
        }
    }
}
```

```

else if(cmd[0] == 'E') {
    scanf("%d", &x);
    int t = team[x];
    if(q2[t].empty()) q.push(t); //团队 t 进入队列
    q2[t].push(x);
}
}
printf("\n");
return 0;
}

```

优先队列是一种抽象数据类型（Abstract Data Type, ADT），行为有些像队列，但先出队列的元素不是先进队列的元素，而是队列中优先级最高的元素，这样就可以允许类似于“急诊病人插队”这样的事情发生。

STL 的优先队列也定义在头文件<queue>里，用“priority_queue<int> pq”来声明。这个 pq 是一个“越小的整数优先级越低的优先队列”。由于出队元素并不是最先进队的元素，出队的方法由 queue 的 front()变为了 top()。

自定义类型也可以组成优先队列，但必须为每个元素定义一个优先级。这个优先级并不需要一个确定的数字，只需要能比较大小即可。看到这里，是不是想起了 sort？没错，只要元素定义了“小于”运算符，就可以使用优先队列。在一些特殊的情况下，需要使用自定义方式比较优先级，例如，要实现一个“个位数大的整数优先级反而小”的优先队列，可以定义一个结构体 cmp，重载“()”运算符，使其“看上去”像一个函数^①，然后用“priority_queue<int, vector<int>, cmp> pq”的方式定义。下面是这个 cmp 的定义：

```

struct cmp {
    bool operator() (const int a, const int b) const { //a 的优先级比 b 小时返回
true
        return a % 10 > b % 10;
    }
};

```

对于一些常见的优先队列，STL 提供了更为简单的定义方法，例如，“越小的整数优先级越大的优先队列”可以写成“priority_queue<int, vector<int>, greater<int>> pq”。注意，最后两个“>”符号不要写在一起，否则会被很多（但不是所有）编译器误认为是“>>”运算符。

提示 5-16: STL 的 queue 头文件提供了优先队列，用“priority_queue<int> s”方式定义，用 push()和 pop()进行元素的入队和出队操作，top()取队首元素（但不删除）。

^① 在 C++ 中，重载了“()”运算符的类或结构体叫做仿函数（functor）。

例题 5-7 丑数 (Ugly Numbers, UVa 136)

丑数是指不能被 2, 3, 5 以外的其他素数整除的数。把丑数从小到大排列起来，结果如下：

1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, ...

求第 1500 个丑数。

【分析】

本题的实现方法有很多种，这里仅提供一种，即从小到大生成各个丑数。最小的丑数是 1，而对于任意丑数 x ， $2x$ 、 $3x$ 和 $5x$ 也都是丑数。这样，就可以用一个优先队列保存所有已生成的丑数，每次取出最小的丑数，生成 3 个新的丑数。唯一需要注意的是，同一个丑数有多种生成方式，所以需要判断一个丑数是否已经生成过。代码如下：

```
#include<iostream>
#include<vector>
#include<queue>
#include<set>
using namespace std;
typedef long long LL;
const int coeff[3] = {2, 3, 5};

int main() {
    priority_queue<LL, vector<LL>, greater<LL> > pq;
    set<LL> s;
    pq.push(1);
    s.insert(1);
    for(int i = 1; ; i++) {
        LL x = pq.top(); pq.pop();
        if(i == 1500) {
            cout << "The 1500'th ugly number is " << x << ".\n";
            break;
        }
        for(int j = 0; j < 3; j++) {
            LL x2 = x * coeff[j];
            if(!s.count(x2)) { s.insert(x2); pq.push(x2); }
        }
    }
    return 0;
}
```

答案：859963392。

5.2.6 测试 STL

和自己写的代码一样，库也是需要测试的。一方面是因为库也是人写的，也有可能

bug, 另一方面是因为测试之后能更好地了解库的用法和优缺点。

提示 5-17: 库不一定没有 bug, 使用之前测试库是一个好习惯。

测试的方法大同小异, 下面只以 `sort` 为例进行介绍。首先, 写一个简单的测试程序:

```
int a[] = {3,2,4};
sort(a, a+3);
printf("%d %d %d\n", a[0], a[1], a[2]);
```

输出为 2 3 4, 是一个令人满意的结果。但这样就够了吗? 不! 测试程序太简单, 说明不了问题。应该写一个更加通用的程序, 随机生成很多整数, 然后排序。

为了随机生成整数, 先来看看随机数发生器。核心函数是 `cstdlib` 中的 `rand()`, 它生成一个闭区间 $[0, \text{RAND_MAX}]$ 内的均匀随机整数 (均匀的含义是: 该区间内每个整数被随机获取的概率相同), 其中 `RAND_MAX` 至少为 32767 ($2^{15}-1$), 在不同环境下的值可能不同。严格地说, 这里的随机数是“伪随机数”, 因为它也是由数学公式计算出来的, 不过在算法领域, 多数情况下可以把它当作真正的随机数。

如何产生 $[0, n]$ 之间的整数呢? 很多人喜欢用 `rand()%n` 产生区间 $[0, n-1]$ 内的一个随机整数, 姑且不论这样产生的整数是否仍然分布均匀, 只要 n 大于 `RAND_MAX`, 此法就不能得到期望的结果。由于 `RAND_MAX` 很有可能只有 32767 这么小, 在使用此法时应当小心。另一个方法是执行 `rand()` 之后先除以 `RAND_MAX`, 得到 $[0, 1]$ 之间的随机实数, 扩大 n 倍后四舍五入, 得到 $[0, n]$ 之间的均匀整数。这样, 在 n 很大时“精度”不好 (好比把小图放大后会看到“锯齿”), 但对于普通的应用, 这样做已经可以满足要求了^①。

提示 5-18: `cstdlib` 中的 `rand()` 可生成闭区间 $[0, \text{RAND_MAX}]$ 内均匀分布的随机整数, 其中 `RAND_MAX` 至少为 32767。如果要生成更大的随机整数, 在精度要求不太高的情况下可以用 `rand()` 的结果“放大”得到。

需要随机数的程序在最开始时一般会执行一次 `srand(time(NULL))`, 目的是初始化“随机数种子”。简单地说, 种子是伪随机数计算的依据。种子相同, 计算出来的“随机数”序列总是相同。如果不调用 `srand` 而直接使用 `rand()`, 相当于调用过一次 `srand(1)`, 因此程序每次执行时, 将得到同一套随机数。

不要在同一个程序每次生成随机数之前都重新调用一次 `srand`。有的初学者抱怨“`rand()` 产生的随机数根本不随机, 每次都相同”, 就是因为误解了 `srand` 的作用。再次强调, 请只在程序开头调用一次 `srand`, 而不要在同一个程序中多次调用。

提示 5-19: 可以用 `cstdlib` 中的 `srand` 函数初始化随机数种子。如果需要程序每次执行时使用一个不同的种子, 可以用 `ctime` 中的 `time(NULL)` 为参数调用 `srand`。一般来说, 只在程序执行的开头调用一次 `srand`。

“同一套随机数”可能是好事也可能是坏事。例如, 若要反复测试程序对不同随机数

^① 如果坚持需要更高的精度, 可以采取多次随机的方法。

据的响应，需要每次得到的随机数不同。一个简单的方法是使用当前时间 `time(NULL)`（在 `ctime` 中）作为参数调用 `srand`。由于时间是不断变化的，每次运行时，一般会得到一套不同的随机数。之所以说“一般会”，是因为 `time` 函数返回的是自 UTC 时间 1970 年 1 月 1 日 0 点以来经过的“秒数”，因此每秒才变化一次。如果你的程序是由操作系统自动批量执行的，可能因为每次运行的间隔时间过短，导致在相邻若干次执行时 `time` 的返回值全部相同。一个解决办法是在测试程序的主函数中设置一个循环，做足够多次测试后再退出^①。

另一方面，如果发现某程序对于一组随机数据报错，就需要在调试时“重现”这组数据。这时，“每次相同的随机序列”就显得十分重要了。不同的编译器计算随机数的方法可能不同。如果是不同编译器编译出来的程序，即使是用相同的参数调用 `srand()`，也可能得到不同的随机序列。

讲了这么多，下面可以编写随机程序了：

```
void fill_random_int(vector<int>& v, int cnt) {
    v.clear();
    for(int i = 0; i < cnt; i++)
        v.push_back(rand());
}
```

注意 `srand` 函数是在主程序开始时调用，而不是每次测试时调用。参数是 `vector<int>` 的引用。为什么不把这个 `v` 作为返回值，而要写到参数里呢？答案是：避免不必要的值被复制。如果这样写：

```
vector<int> fill_random_int(int cnt) {
    vector<int> v;
    for(int i = 0; i < cnt; i++)
        v.push_back(rand());
    return v;
}
```

实际上函数内的局部变量 `v` 中的元素需要逐个复制给调用者。而用传引用的方式调用，就避免了这些复制过程。

提示 5-20：把 `vector` 作为参数或者返回值时，应尽量改成用引用方式传递参数，以避免不必要的值被复制。

这两个函数可以同时存在于一份代码中，因为 C++ 支持函数重载，即函数名相同但参数不同的两个函数可以同时存在。这样，编译器可以根据函数调用时参数类型的不同判断应该调用哪个函数。如果两个函数的参数相同^②只是返回值不同，是不能重载的。

提示 5-21：C++ 支持函数重载，但函数的参数类型必须不同（不能只有返回值类型不同）。

^① 还有一个更通用的方法将在附录 A 中说明。

^② 准确地说，应该是参数类型相同，参数的名字是无关紧要的。